Meetup Python Rennes

Python At Scale

A Reech Case Study

2022-04-07

About Me

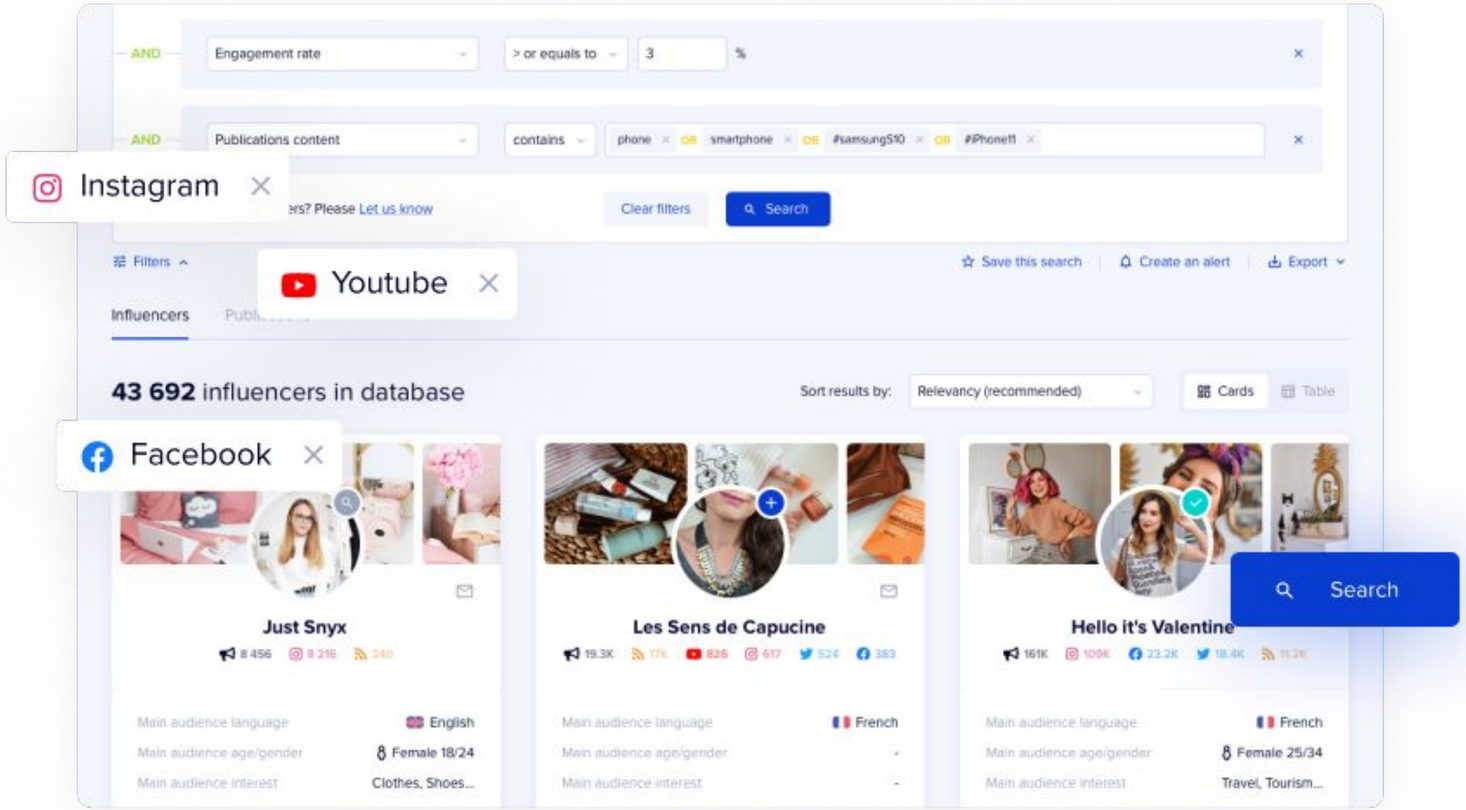
- Michel Caradec
  - michel@reech.com
- Lead Data Engineer At Reech
- Former Software Engineer
  - Always played with data & databases
    - Relational
    - Multidimensional
    - NoSQL
    - Graph
  - Distributed environments and Cloud computing
- Member of Breizh Data Club
  https://breizhdataclub.org/

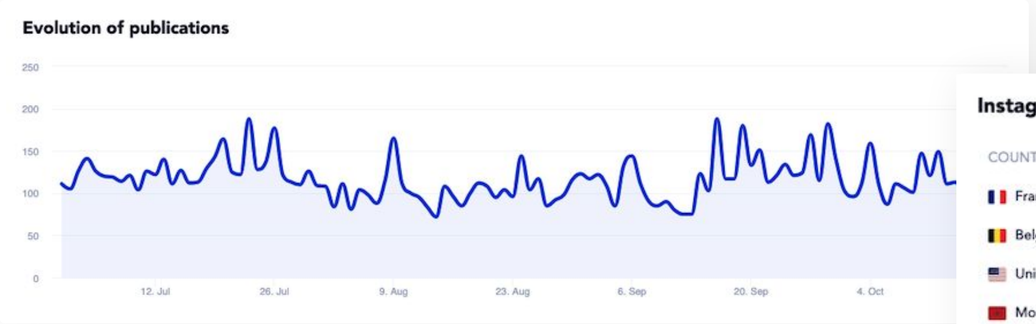
About Reech

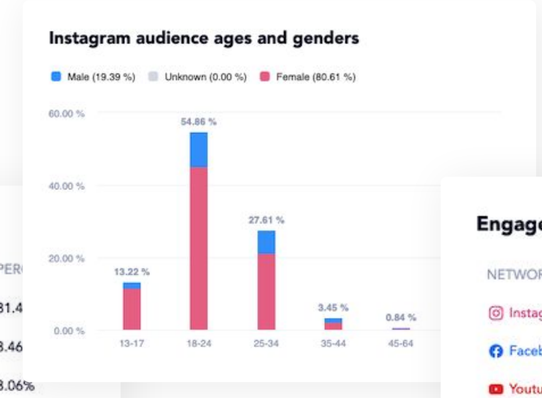
- Marketing influence expert since 2015
  - 60 experts
  - 150 customers
  - 1000 campaigns
  - Group Dékuple
- What is Marketing influence?
- 2 offers
  - Reech Agency: marketing strategy 💫
  - Reech Influence Cloud: software 🤓
- More at
  - https://www.welcometothejungle.com/fr/companies/reech/
  - https://reech.tech/

Reech Influence Cloud - Since 2020

- Search ambassadors
  - 200+ search filters
- Multiple social networks
- 8 millions of influencers
- Some customers
  - Oppo, Philips, Square Enix
  - CRT Bretagne







NETWORK	MIN	MAX	AVG
Instagram	0.00%	80.06%	4.42%
Facebook	0.00%	5.88%	0.57%
Youtube	2.44%	21.40%	7.50%
Pinterest	0.01%	9.62%	0.38%
Twitter	0.01%	328.21%	0.79%

About Data Collection

Reech **code of conduct**

- No **personal** account collection
- No **private data** on influencers nor followers

- Only **business** accounts on Instagram
- Minimum of **10 000** followers

- No **scraping**

The Problem - At Local Scale

Worker ⚙️

Schedule ⚡ → Collect → Transform → Store

- Schedule the collect operations

- Collect the latest updates from the social networks by invoking their APIs - *Python module* ***requests***

- Process the result returned by the API (mainly JSON) - *Python module* ***json***
- Extract relevant information (#hastags, @mentions) - *Python module* ***re***
- Enrich data
  - Language detection
  - Sentiment analysis

- Persist the result

Demonstration - Step 1 - Monolith

Project structure:

```
src/
  business/
    profile.py      <= Influencer profile management
    crud.py         <= Storage manipulation
  utils/
  step_XX_*/
    scheduler.py    <= The scheduler (plan things)
    worker.py       <= The worker (collect, transform & store)
  launcher.py       <= There must be some events to trigger the operations
docker/
log/
```

Repository: https://gitlab.com/reech-oss/meetup/meetup-python-rennes-at-scale

Monolith Caveats

- **Sequential** processing
  - One task after another
- Does not **scale**…

Monolith

```
def scheduler(social_type: str):
    for profile in get_candidate_profiles(social_type):
        profile.update()
```

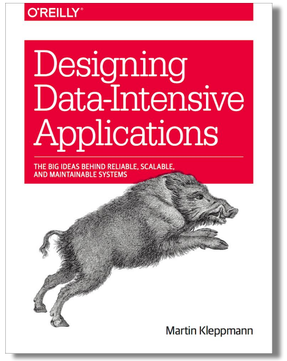
reech.com

The Problem - At Real Scale

- Must support **numerous** influencers / publications
  - 8 millions of influencers
- **Frequency** of refresh
  - Daily / monthly
- **Error** handling
  - Non responding API
  - Network failure
- API **throttling**
  - Rate limit

"In distributed system, suspicion, pessimism and paranoia pay off"
Martin Kleppmann - Designing Data-Intensive Applications
https://dataintensive.net/

https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing



Demonstration - Step 2 - Multithreading

A first attempt to address our problem

Multithreading Caveats

Multithreading

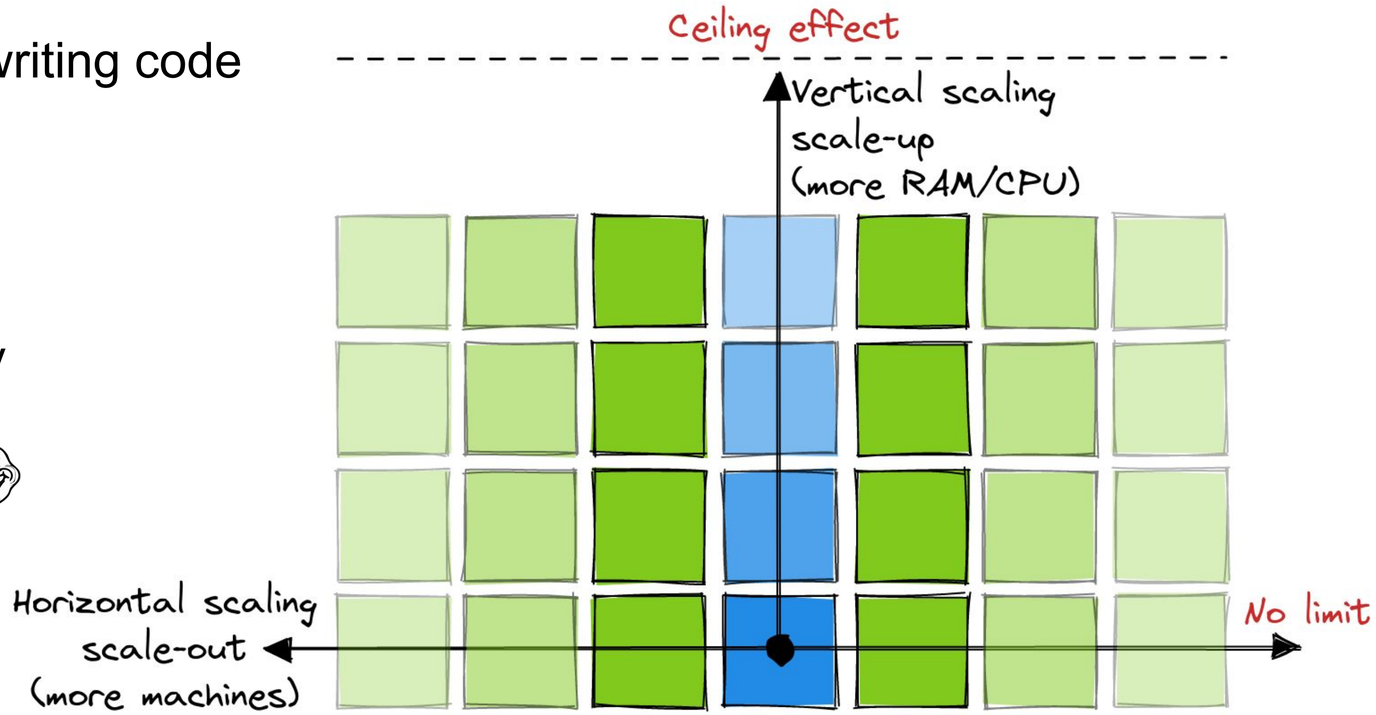
```python
from concurrent.futures import ThreadPoolExecutor

def scheduler(social_type: str):
    with ThreadPoolExecutor() as pool:
        for profile in get_candidate_profiles(social_type):
            pool.submit(worker, profile)

def worker(profile: Profile):
    profile.update(in_error=in_error)
```

reech.com

Multithreading Caveats

- We must keep in **mind** multithreading while writing code
  - CPUs, Threads
- In case of **exception**
  - Some **state** must be saved
  - Some **retry logic** must be implemented
- Will only **scale vertically**, but not horizontally

- Python is not very good at multithreading
  - Global Interpreter Lock



The Solution - Cloud Design Patterns

We will mention a few, and demonstrate them

- **Queue-Based Load Leveling**
  - Keep **independent** execution units
    - Submitter **guaranteed** the task will be executed, with **no blocking**
    - Receiver **free** to execute the task based on its capacity
  - Smooth intermittent heavy loads

- **Retry-Logic**
  - Handle **transient failures** when connecting to a service
  - **Transparently** retry a failed operation

- **Throttling**
  - Control the consumption of resources

More on https://docs.microsoft.com/en-us/azure/architecture/patterns/, https://martinfowler.com/architecture/

Demonstration - Steps 3-5 - Messaging

Use of **Celery**

- Package https://pypi.org/project/celery/
- Provides task queues as a mechanism to **distribute** work across **threads** or **machines**
  - Horizontal scaling
- Communication via **messages**
- Use of a **broker** to mediate between **clients** and **workers**
  - **RabbitMQ**
  - https://www.rabbitmq.com/

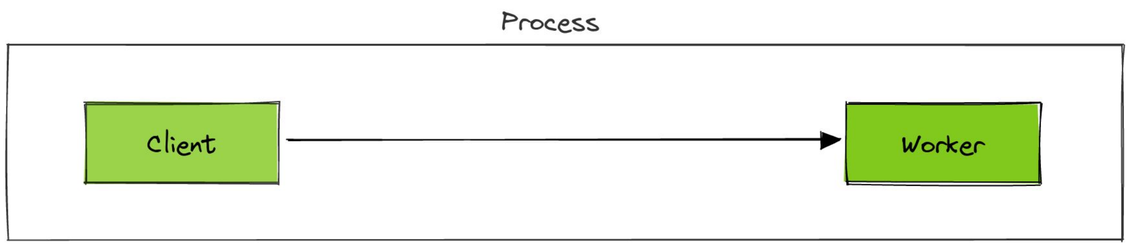
Wrap Up - What We Did

Messaging

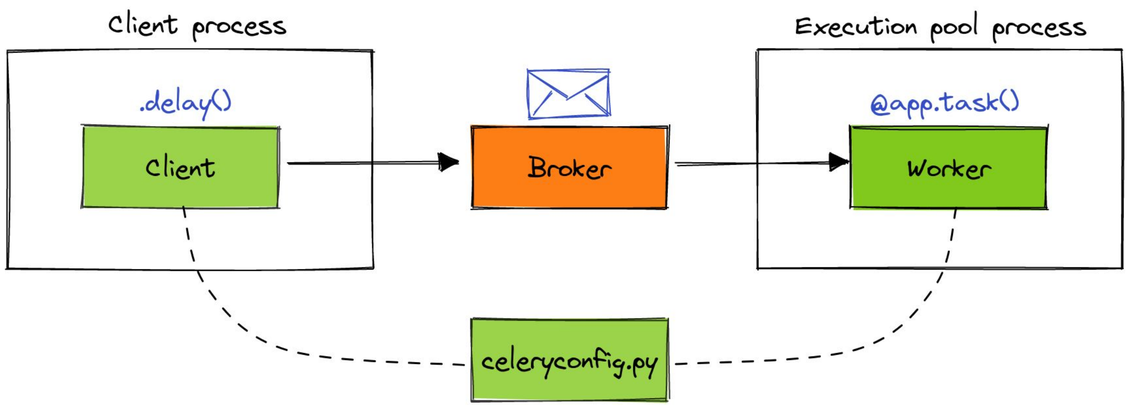
```python
app = Celery()

def scheduler(social_type: str):
    for profile in get_candidate_profiles(social_type):
        worker.delay(profile.social_type, profile.social_id, profile.slug)

@app.task(name=TASK_CONSUMER_NAME, queue=QUEUE_NAME)
def worker(social_type: str, social_id: str, slug: str):
    profile = Profile(social_type, social_id, slug)
    profile.update()
```

reech.com

Wrap Up - What We Did



- **Orchestration** using message based communication
- Use of **Celery**, with `@decorators` and `.delay()` calls
  - No effect on business implementation
  - Function oriented
- Use of **RabbitMQ** as messaging service



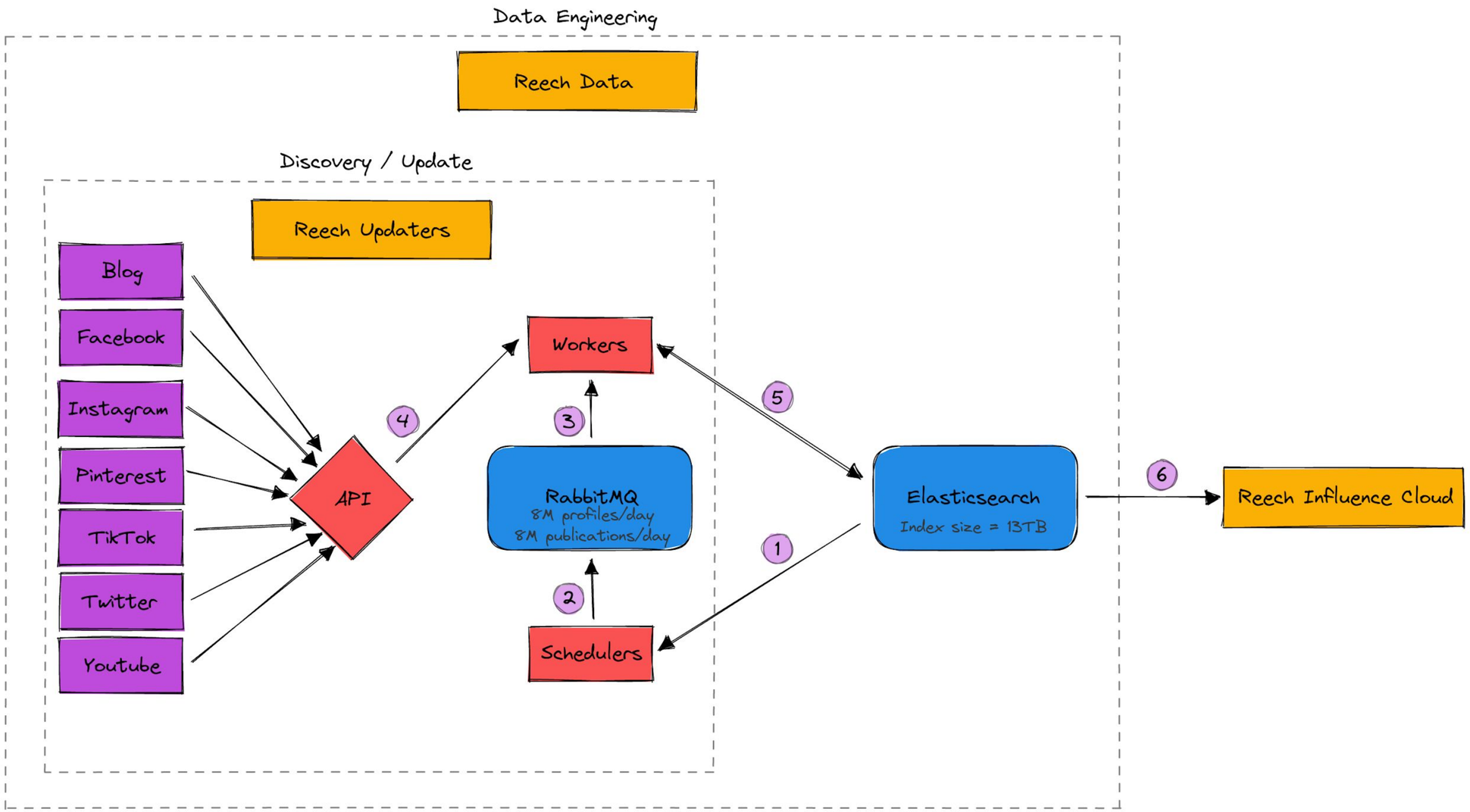
Wrap Up - What We Got

- **Loose coupling** between clients and workers
- Control on **concurrency**
  - Manual scaling
  - Auto-scaling
- Control on **velocity**
  - Rate limiting
- **Retry logic** mechanism
  - Dead letter

Cloud Design Patterns At Reech - Facts

- **13 TB** of data
- Every day
  - **8 millions** profiles processed
  - 8 millions publications ingested/updated
- **80** message queues
- **1750** workers running on multiple servers on steroids

Reech - The Big Picture



Why Python?

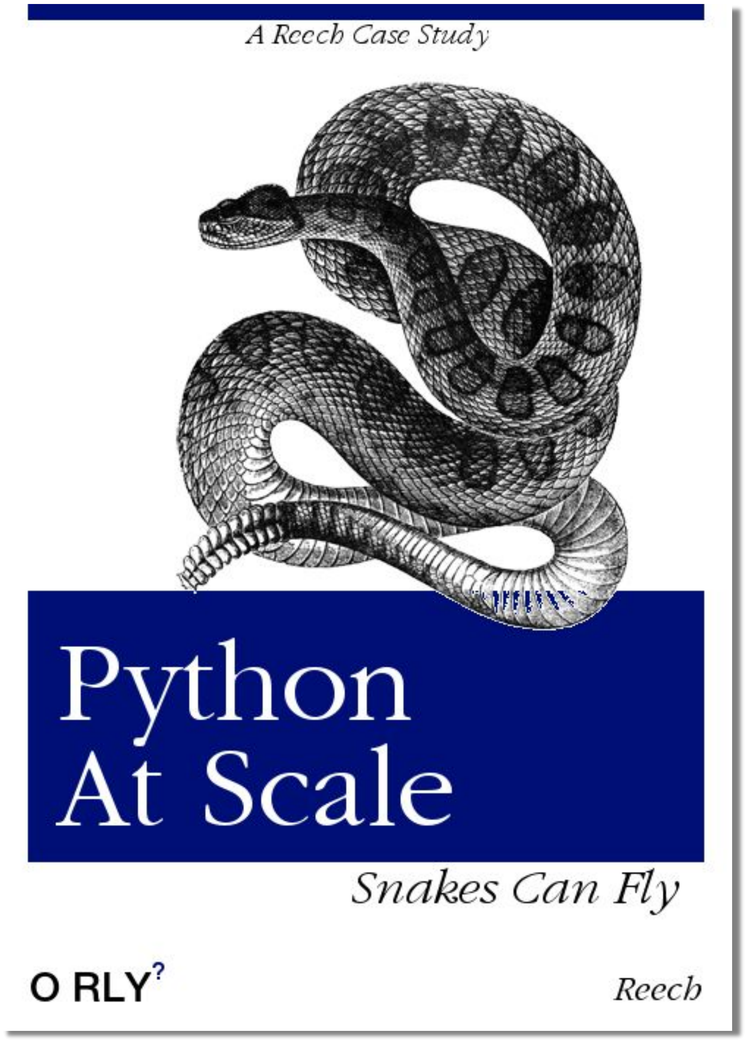
- No **complex** code
  - Mainly <u>function oriented</u>
- No advanced **object oriented** features required
  - Classes, inheritance
- Python is easy to **write**, easy to **maintain**
- Python has good packages **ecosystem**
  - *There is always a Python package for this*
- Python is **multi-platform**
- **Performance** is not a key requirement
  - At least compared to network latency
  - ⚠️ Consider heavy computation requirements
    - Machine Learning
    - Interpreted vs compiled language

Reech Ops

- Deployment
  - Initial: Terraform/Ansible => part of the disaster recovery plan
  - Continuous: Gitlab
- Monitoring
  - Infrastructure: Prometheus + Grafana
  - Applications: Filebeat + Kibana (module **logger**)
  - Data quality (package **elastalert**)

There Is A Lot More

But that’s another story, for future Meetups 🤓



This book is not for sale!

Thank You

Questions & Answers